

Lab 7: Stored Procedure (PL/SQL)

Use "lab7 Files" folder

- Save the file and extract it under Lab folder
- Remember to start by creating a silicon database to use.

A. Stored Procedure

A0. Basic of Stored Procedure

Why? Stored procedures are fast. MySQL server takes some advantage of caching, just as prepared statements do. The main speed gain comes from reduction of network traffic. If you have a repetitive task that requires checking, looping, multiple statements, and no user interaction, do it with a single call to a procedure that's stored on the server.

Before creating a procedure:

- **Check MySQL version:**

```
mysql>SELECT VERSION();
```

- Check the **privileges** of the current user :

```
mysql> SHOW PRIVILEGES;
```

Need CREATE ROUTINE privilege, and CREATE FUNCTION might require the SUPER privilege

- Create 'hr' database:

```
mysql> source hr-schema-mysql.sql;
```

- Pick a **Delimiter**: The delimiter is the character or string of characters which is used to complete an SQL statement. By default we use semicolon (;) as a delimiter. But this causes problem in stored procedure because a procedure can have many statements, and everyone must end with a semicolon. So for your delimiter, pick a string which is rarely occur within statement or within procedure. Here we have used double dollar sign i.e. \$\$.

```
mysql> DELIMITER $$ ;
```

- DELIMITER is set to it's default.

```
mysql> DELIMITER ; $$
```

A1. Creating simple procedure

Query 1: Simple procedure

```
mysql> DELIMITER $$ ;
mysql> CREATE PROCEDURE job_data()
-> SELECT * FROM jobs; $$
Query OK, 0 rows affected (0.05 sec)
mysql> DELIMITER ; $$
```

A2. Executing procedure

Query 2.

```
mysql> call job_data;
```

job_id	job_title	min_salary	max_salary
AC_ACCOUNT	Public Accountant	4200	9000
AC_MGR	Accounting Manager	8200	16000
AD_ASST	Administration Assistant	3000	6000
AD PRES	President	20000	40000
AD_VP	Administration Vice President	15000	30000
FI_ACCOUNT	Accountant	4200	9000
FI_MGR	Finance Manager	8200	16000
HR_REP	Human Resources Representative	4000	9000
IT_PROG	Programmer	4000	10000
MK_MAN	Marketing Manager	9000	15000
MK_REP	Marketing Representative	4000	9000
PR_REP	Public Relations Representative	4500	10500
PU_CLERK	Purchasing Clerk	2500	5500
PU_MAN	Purchasing Manager	8000	15000
SA_MAN	Sales Manager	10000	20000
SA_REP	Sales Representative	6000	12000
SH_CLERK	Shipping Clerk	2500	5500
ST_CLERK	Stock Clerk	2000	5000
ST_MAN	Stock Manager	5500	8500

A3. Show create procedure

This statement is a MySQL extension. It returns the exact string that can be used to re-create the named stored procedure. Both statement require that you be the owner of the routine.

Query 3.

```
mysql> SHOW CREATE PROCEDURE job_data;
```

Procedure	sql_mode	character_set_client	collation_connection	Database Collation
job_data	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_F			

```

OR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION | CREATE
DEFINER=`root`@`%` PROCEDURE `job_data`()
SELECT * FROM jobs | cp850 | cp850_general_ci |
utf8_general_ci |
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
1 row in set (0.10 sec)

```

A4. MySQL : Compound-Statement

A compound statement is a block that can contain other blocks; declarations for variables, condition handlers, and cursors; and flow control constructs such as loops and conditional tests.

As of version 5.6 MySQL have following compound statements :

- BEGIN ... END Compound-Statement
- Statement Label
- DECLARE
- Variables in Stored Programs
- Flow Control Statements
- Cursors
- Condition Handling

Query 4.

```

mysql> DELIMITER $$
mysql> CREATE PROCEDURE my_procedure_Local_Variables()
-> BEGIN /* declare local variables */
-> DECLARE a INT DEFAULT 10;
-> DECLARE b, c INT; /* using the local variables */
-> SET a = a + 100;
-> SET b = 2;
-> SET c = a + b;
-> BEGIN /* local variable in nested block */
-> DECLARE c INT;
-> SET c = 5;
-> /* local variable c takes precedence over the one of the
/*> same name declared in the enclosing block. */
-> SELECT a, b, c;
-> END;
-> SELECT a, b, c;
-> END$$

```

Query OK, 0 rows affected (0.05 sec)

```

mysql> DELIMITER ;
mysql> call my_procedure_Local_Variables;
+-----+-----+-----+
| a      | b      | c      |
+-----+-----+-----+
| 110    | 2       | 5       |
+-----+-----+-----+
1 row in set (0.04 sec)

```

```

+-----+-----+-----+
| a      | b      | c      |

```

```

+-----+-----+-----+
| 110 | 2 | 112 |
+-----+-----+-----+
1 row in set (0.04 sec)

```

Query OK, 0 rows affected (0.09 sec)

A5. User variables

User variables are referenced with an ampersand (@) prefixed to the user variable name (for example, @x and @y). The following example shows the use of user variables within the stored procedure :

Query 5.

```

mysql> DELIMITER $$
mysql> CREATE PROCEDURE my_procedure_User_Variables()
-> BEGIN
-> SET @x = 15;
-> SET @y = 10;
-> SELECT @x, @y, @x-@y;
-> END$$

```

Query OK, 0 rows affected (0.08 sec)

```

mysql> DELIMITER ;
mysql> CALL my_procedure_User_Variables();
+-----+-----+-----+
| @x | @y | @x-@y |
+-----+-----+-----+
| 15 | 10 | 5 |
+-----+-----+-----+
1 row in set (0.06 sec)

```

Query OK, 0 rows affected (0.06 sec)

A6. Procedure Parameters

We can divide the CREATE PROCEDURE statement in the following ways :

0. CREATE PROCEDURE sp_name () ...
- A. CREATE PROCEDURE sp_name ([IN] param_name type)...
- B. CREATE PROCEDURE sp_name ([OUT] param_name type)...
- C. CREATE PROCEDURE sp_name ([INOUT] param_name type)...

Query 6A. Parameter IN example

```

mysql> DELIMITER $$
mysql> CREATE PROCEDURE my_proc_IN (IN var1 INT)
-> BEGIN
-> SELECT * FROM jobs LIMIT var1;
-> END$$

```

Query OK, 0 rows affected (0.05 sec)

```
mysql> DELIMITER ;
```

% To execute the first 2 rows from the 'jobs' table

```
mysql> CALL my_proc_in(2);
```

job_id	job_title	min_salary	max_salary
AC_ACCOUNT	Public Accountant	4200	9000
AC_MGR	Accounting Manager	8200	16000

2 rows in set (0.05 sec)

Query OK, 0 rows affected (0.06 sec)

% To execute the first 5 rows from the 'jobs' table

mysql> CALL my_proc_in(5);

job_id	job_title	min_salary	max_salary
AC_ACCOUNT	Public Accountant	4200	9000
AC_MGR	Accounting Manager	8200	16000
AD_ASST	Administration Assistant	3000	6000
AD PRES	President	20000	40000
AD_VP	Administration Vice President	15000	30000

5 rows in set (0.04 sec)

Query OK, 0 rows affected (0.05 sec)

Query 6B. Parameter OUT example

mysql> DELIMITER \$\$

mysql> CREATE PROCEDURE my_proc_OUT (OUT highest_salary INT)

-> BEGIN

-> SELECT MAX(MAX_SALARY) INTO highest_salary FROM jobs;

-> END\$\$

Query OK, 0 rows affected (0.05 sec)

sql> DELIMITER ;

mysql> CALL my_proc_OUT(@M);

Query OK, 1 row affected (0.05 sec)

mysql> SELECT @M\$\$

@M
40000

1 row in set (0.05 sec)

Query 6C. Parameter INOUT example

mysql> DELIMITER \$\$

mysql> CREATE PROCEDURE my_proc_INOUT (INOUT empcnt INT, IN dept_id INT)

-> BEGIN

-> SELECT COUNT(employee_id) INTO empcnt FROM employees WHERE
department_id = dept_id;

-> END\$\$

Query OK, 0 rows affected (0.08 sec)

mysql> DELIMITER ;

```
mysql> CALL my_proc_INOUT(@C,10);
Query OK, 1 row affected (0.05 sec)
```

```
mysql> SELECT @C;
+-----+
| @C    |
+-----+
|      1 |
+-----+
1 row in set (0.05 sec)
```

```
mysql> CALL my_proc_INOUT(@C,100);
Query OK, 1 row affected (0.05 sec)
```

```
mysql> SELECT @C;;
+-----+
| @C    |
+-----+
|      6 |
+-----+
1 row in set (0.05 sec)
```

A7. Flow Control Statements

Query 7. If Statement

```
IF condition THEN statement(s)
[ELSEIF condition THEN statement(s)] ...
[ELSE statement(s)]
END IF
```

% In the following example, we pass user_id through IN parameter to get the user name. Within the procedure, we have used IF ELSEIF and ELSE statement to get user name against multiple user id. The user name will be stored into INOUT parameter user_name.

```
mysql> DELIMITER $$

mysql> CREATE PROCEDURE GetUserName(INOUT user_name varchar(16),
-> IN user_id varchar(16))
-> BEGIN
-> DECLARE uname varchar(16);
-> SELECT name INTO uname
-> FROM user
-> WHERE userid = user_id;
-> IF user_id = "scott123"
-> THEN
-> SET user_name = "Scott";
-> ELSEIF user_id = "ferp6734"
-> THEN
-> SET user_name = "Palash";
-> ELSEIF user_id = "diana094"
-> THEN
-> SET user_name = "Diana";
-> END IF;
-> END$$
```

```

Query OK, 0 rows affected (0.05 sec)
mysql> DELIMITER ;

mysql> CALL GetUserName(@A,'scott123');
Query OK, 1 row affected (0.03 sec)

mysql> SELECT @A;
+-----+
| @A    |
+-----+
| Scott |
+-----+

```

Query 8. Case Statement

```

CASE case_value
WHEN when_value THEN statement_list
[WHEN when_value THEN statement_list] ...
[ELSE statement_list] END CASE
or
CASE
WHEN search_condition THEN statement_list
[WHEN search_condition THEN statement_list] ...
[ELSE statement_list] END CASE

```

% We want to count the number of employees with following conditions :

- MIN_SALARY > 10000
- MIN_SALARY < 10000
- MIN_SALARY = 10000

```

mysql> DELIMITER $$

mysql> CREATE PROCEDURE my_proc_CASE
-> (INOUT no_employees INT, IN salary INT)
-> BEGIN
-> CASE
-> WHEN (salary>10000)
-> THEN (SELECT COUNT(job_id) INTO no_employees
-> FROM jobs
-> WHERE min_salary>10000);
-> WHEN (salary<10000)
-> THEN (SELECT COUNT(job_id) INTO no_employees
-> FROM jobs
-> WHERE min_salary<10000);
-> ELSE (SELECT COUNT(job_id) INTO no_employees
-> FROM jobs WHERE min_salary=10000);
-> END CASE;
-> END$$

Query OK, 0 rows affected (0.19 sec)
mysql> DELIMITER ;

```

% Number of employees whose salary greater than 10000 :

```

mysql> CALL my_proc_CASE(@C,10001);
Query OK, 1 row affected (1.39 sec)

mysql> SELECT @C;

```

```

+-----+
| @C    |
+-----+
|      2 |
+-----+
1 row in set (0.23 sec)

```

% Number of employees whose salary less than 10000 :
mysql> CALL my_proc_CASE(@C,9999);
Query OK, 1 row affected (0.22 sec)

```

mysql> SELECT @C;
+-----+
| @C    |
+-----+
|     16 |
+-----+
1 row in set (1.00 sec)

```

% Number of employees whose salary equal to 10000 :
mysql> CALL my_proc_CASE(@C,10000);
Query OK, 1 row affected (1.57 sec)

```

mysql> SELECT @C;
+-----+
| @C    |
+-----+
|      1 |
+-----+
1 row in set (0.05 sec)

```

Query 9. Loop Statement

```

[begin_label:]
LOOP
statement_list
END LOOP
[end_label]

```

% In the following procedure rows will be inserted in 'number' table until x is less than num (number supplied by the user through IN parameter). A random number will be stored every time.

```

mysql> DROP TABLE number;
Query OK, 0 rows affected (1.15 sec)

mysql> CREATE TABLE number (num decimal);
Query OK, 0 rows affected (0.06 sec)

mysql> DELIMITER $$
mysql> CREATE PROCEDURE my_proc_LOOP (IN num INT)
-> BEGIN
-> DECLARE x INT;
-> SET x = 0;
-> loop_label: LOOP
-> INSERT INTO number VALUES (rand());
-> SET x = x + 1;
-> IF x >= num

```



```

-> THEN
-> LEAVE loop_label;
-> END IF;
-> END LOOP;
-> END$$
ERROR 1304 (42000): PROCEDURE my_proc_LOOP already exists
mysql> DELIMITER ;

```

```

mysql> CALL my_proc_LOOP(3);
Query OK, 1 row affected (0.09 sec)

```

```

mysql> select * from number;
+-----+
| num |
+-----+
|    0 |
|    1 |
|    0 |
+-----+

```

Return Statement

The RETURN statement terminates execution of a stored function and returns the value expr to the function caller. There must be at least one RETURN statement in a stored function. There may be more than one if the function has multiple exit points. Here is the syntax :

```
RETURN expr
```

Query 10. While Statement

The WHILE statement executes the statement(s) as long as the condition is true. The condition is checked every time at the beginning of the loop. Each statement is terminated by a semicolon (;). Here is the syntax :

```

[begin_label:] WHILE search_condition DO
    statement_list
END WHILE [end_label]

```

```
mysql> DELIMITER $$
```

```

mysql> CREATE PROCEDURE my_proc_WHILE(IN n INT)
-> BEGIN
-> SET @sum = 0;
-> SET @x = 1;
-> WHILE @x<n
-> DO
->     IF mod(@x, 2) <> 0 THEN
-> SET @sum = @sum + @x;
-> END IF;
-> SET @x = @x + 1;
-> END WHILE;
-> END$$

```

```

Query OK, 0 rows affected (0.37 sec)
mysql> DELIMITER $$

```

```

mysql> CALL my_proc_WHILE(5);
Query OK, 0 rows affected (0.05 sec)

```

```

mysql> SELECT @sum;
+-----+

```

```

| @sum |
+-----+
|      4 |
+-----+
1 row in set (0.05 sec)

mysql> CALL my_proc_WHILE(10);
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT @sum;
+-----+
| @sum |
+-----+
|     25 |
+-----+
1 row in set (0.05 sec)

```

A8. Alter and Drop Statements

Query 11. ALTER Procedure

```

ALTER PROCEDURE proc_name [characteristic ...]characteristic:
COMMENT 'string'
| LANGUAGE SQL
| { CONTAINS SQL
| NO SQL | READS SQL DATA
| MODIFIES SQL DATA }
| SQL SECURITY { DEFINER
| INVOKER }

```

```

mysql> ALTER PROCEDURE my_proc_WHILE
-> COMMENT 'Modify Comment';

```

Query 12. DROP Procedure

```

DROP {PROCEDURE | FUNCTION} [IF EXISTS] sp_name

```

```

mysql> DROP PROCEDURE my_proc_WHILE;
Query OK, 0 rows affected (0.05 sec)

```

A9. CURSOR

A database cursor is a control structure that enables traversal over the records in a database. Cursors are used by database programmers to process individual rows returned by database system queries. Cursors enable manipulation of whole result sets at once. In this scenario, a cursor enables the rows in a result set to be processed sequentially. In SQL procedures, a cursor makes it possible to define a result set (a set of data rows) and perform complex logic on a row by row basis. By using the same mechanics, an SQL procedure can also define a result set and return it directly to the caller of the SQL procedure or to a client application.

MySQL supports cursors inside stored programs. The syntax is as in embedded SQL. Cursors have these properties :

- Asensitive: The server may or may not make a copy of its result table
- Read only: Not updatable
- Nonscrollable: Can be traversed only in one direction and cannot skip rows

To use cursors in MySQL procedures, you need to do the following :

- Declare a cursor.
- Open a cursor.
- Fetch the data into variables.
- Close the cursor when done.

Declare a cursor:

The following statement declares a cursor and associates it with a SELECT statement that retrieves the rows to be traversed by the cursor.

```
DECLARE cursor_name  
CURSOR FOR select_statement
```

Open a cursor :

The following statement opens a previously declared cursor.

```
OPEN cursor_name
```

Fetch the data into variables :

This statement fetches the next row for the SELECT statement associated with the specified cursor (which must be open) and advances the cursor pointer. If a row exists, the fetched columns are stored in the named variables. The number of columns retrieved by the SELECT statement must match the number of output variables specified in the FETCH statement.

```
FETCH [[NEXT] FROM] cursor_name  
INTO var_name [, var_name] ...
```

Close the cursor when done :

This statement closes a previously opened cursor. An error occurs if the cursor is not open.

```
CLOSE cursor_name
```

Query 13. Cursor Example

The procedure starts with three variable declarations. Incidentally, the order is important. First, declare variables. Then declare conditions. Then declare cursors. Then, declare handlers. If you put them in the wrong order, you will get an error message.

```
mysql> DELIMITER $$  
  
mysql> CREATE PROCEDURE my_procedure_cursors(INOUT return_val INT)  
-> BEGIN  
-> DECLARE a,b INT;  
-> DECLARE cur_1 CURSOR FOR  
-> SELECT max_salary FROM jobs;  
-> DECLARE CONTINUE HANDLER FOR NOT FOUND SET b = 1;  
-> OPEN cur_1;  
-> REPEAT FETCH cur_1 INTO a;  
-> UNTIL b = 1  
-> END REPEAT;  
-> CLOSE cur_1;  
-> SET return_val = a;  
-> END;  
-> $$  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> DELIMITER ;
```

```
mysql> CALL my_procedure_cursors(@R);
Query OK, 0 rows affected (0.05 sec)
```

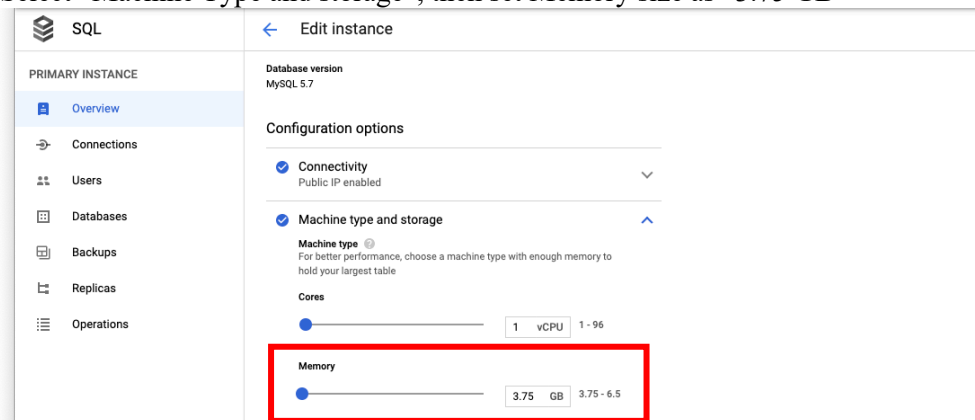
```
mysql> SELECT @R$$
+-----+
| @R    |
+-----+
| 8500  |
+-----+
1 row in set (0.05 sec)
```

B. Trigger in MySQL

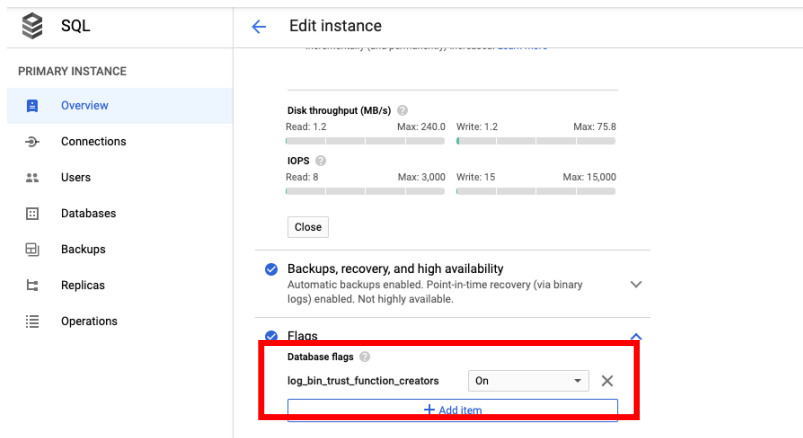
A trigger is a set of actions that are run automatically when a specified change operation (SQL INSERT, UPDATE, or DELETE statement) is performed on a specified table. Triggers are useful for tasks such as enforcing business rules, validating input data, and keeping an audit trail.

Before you move on the next, you should change the configuration of MySQL running on GCP.

- Open GCP console → SQL service → Select MySQL instance → Select Overview
- Fine Configuration tile, then select “→ Edit Configuration”
- Select “Machine Type and storage”, then set Memory size as “3.75 GB”



- Select “Flags”
- Find “log_bin_trust_function_creators” and set “ON”



- Save and restart the instance
- After restart, connect mysql

B1. Syntax

```
CREATE
[DEFINER = { user | CURRENT_USER }]
TRIGGER trigger_name
trigger_time trigger_event
ON tbl_name FOR EACH ROW
trigger_body
trigger_time: { BEFORE | AFTER }
trigger_event: { INSERT | UPDATE | DELETE }
```

Query 14 Simple Example

```
mysql> CREATE TRIGGER ins_sum BEFORE INSERT ON account
-> FOR EACH ROW SET @sum = @sum + NEW.amount;
(error due to missing account table)
```

B2. Example AFTER INSERT

In the following example, we have two tables : emp_details and log_emp_details. To insert some information into log_emp_details table (which have three fields employee id and salary and edtime) every time, when an INSERT happen into emp_details table we have used the following trigger :

Query 15.

```
mysql> use hr;
mysql> CREATE TABLE log_emp_details (
-> emp_details int(11),
-> salary decimal(8,2),
-> edtime datetime);
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TRIGGER emp_details_AINS
-> AFTER INSERT ON employees
-> FOR EACH ROW
-> -- Edit trigger body code below this line. Do not edit lines above
this one
-> BEGIN
```

```

-> INSERT INTO log_emp_details
-> VALUES(NEW.employee_id, NEW.salary, NOW());
-> END$$
Query OK, 0 rows affected (0.17 sec)

mysql> select * from log_emp_details$$
Empty set (0.05 sec)

mysql> INSERT INTO employees VALUES (236,'RABI', 'CHANDRA',
-> 'RABI','590.423.45700',
-> STR_TO_DATE('12-JAN-2013', '%d-%M-%Y'),'AD_VP', 15000,
-> NULL, NULL,90)$$
Query OK, 1 row affected (0.15 sec)

mysql> SELECT * FROM employees WHERE employee_id = 236$$
+-----+-----+-----+-----+-----+-----+
| employee_id | first_name | last_name | email | phone_number | hire_date |
| job_id | salary | commission_pct | manager_id | department_id |
+-----+-----+-----+-----+-----+-----+
| 236 | RABI | CHANDRA | RABI | 590.423.45700 | 2013-01-12 |
| AD_VP | 15000.00 | NULL | NULL | 90 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.05 sec)

mysql> SELECT * FROM log_emp_details$$
+-----+-----+-----+
| emp_details | salary | edittime |
+-----+-----+-----+
| 236 | 15000.00 | 2017-08-22 02:04:34 |
+-----+-----+-----+
1 row in set (0.04 sec)

```

B3. Example BEFORE INSERT

In the following example, before insert a new record in employees table, a trigger check the column value of FIRST_NAME, LAST_NAME, JOB_ID and

- If there are any space(s) before or after the FIRST_NAME, LAST_NAME, TRIM() function will remove those.

- The value of the JOB_ID will be converted to upper cases by UPPER() function.

Query 16.

```
mysql> use hr;
```

```
mysql> DELIMITER $$
```

```

mysql> CREATE TRIGGER emp_details_BINS
-> BEFORE INSERT
-> ON employees FOR EACH ROW
-> -- Edit trigger body code below this line. Do not edit lines above
this one
-> BEGIN
-> SET NEW.FIRST_NAME = TRIM(NEW.FIRST_NAME);

```

```

-> SET NEW.LAST_NAME = TRIM(NEW.LAST_NAME);
-> SET NEW.JOB_ID = UPPER(NEW.JOB_ID);
-> END;$$
Query OK, 0 rows affected (0.23 sec)
mysql> DELIMITER ;

mysql> INSERT INTO employees VALUES (334, ' Ana ', ' King', 'ANA',
-> '690.432.45701', STR_TO_DATE('05-FEB-2013', '%d-%M-%Y'),
-> 'it_prog', 17000, NULL, NULL, 90);
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM employees WHERE employee_id = 334;
+-----+-----+-----+-----+-----+-----+
| employee_id | first_name | last_name | email | phone_number | hire_date |
| job_id | salary | commission_pct | manager_id | department_id |
+-----+-----+-----+-----+-----+-----+
| 334 | Ana | King | ANA | 690.432.45701 | 2013-02-05 |
| IT_PROG | 17000.00 | NULL | NULL | NULL | 90 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.05 sec)

```

Check this:

```

FIRST_NAME - > ' Ana ' has changed to 'Ana'
LAST_NAME - > ' King' has changed to 'King'
JOB_ID - > ' it_prog' has changed to 'IT_PROG'

```

Exercise

Exercise 1: Creating Two Tables

For this exercise, first create a student_mast table including three columns STUDENT_ID, NAME, ST_CLASS.

```
mysql> SELECT * FROM STUDENT_MAST;
```

STUDENT_ID	NAME	ST_CLASS
1	Steven King	7
2	Neena Kochhar	8
3	Lex De Haan	8
4	Alexander Hunold	10

stu_log table has two columns student_id and description.

Create student_mast and stu_log tables.

Exercise 2: Creating Trigger for AFTER UPDATE

Let us promote all the students in next class i.e., 7 will be 8, 8 will be 9 and so on. After updating a single row in student_mast table a new row will be inserted in stu_log table where we will store the current student id and a small description regarding the current update.

- Create a trigger
- Update student_mast table using the following:
- mysql> UPDATE STUDENT_MAST SET ST_CLASS = ST_CLASS + 1;
- Select both student_mast and stu_log tables

Submit a source of trigger and a screenshot to Canvas.